

UD 2. IDENTIFICACIÓN DE LOS ELEMENTOS DE UN PROGRAMA INFORMÁTICO - JAVA -

**Programación – 1º DAM
Curso 2025/2026**

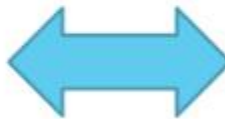
Lenguajes de Programación

- Ya hemos visto que un **algoritmo** es un conjunto ordenado y finito de pasos que nos permiten llegar a la solución de un problema. Cuando ese problema debe ser resuelto por un ordenador el algoritmo se convierte en un **programa**.

PROBLEMA



Algoritmo



Programa

Tipos de lenguaje

Lenguaje natural



Es el lenguaje que utilizamos los humanos para comunicarnos. Por ejemplo, el castellano, el inglés, el francés,...

El lenguaje natural es flexible y nos permite expresar ideas de diferentes maneras, pero también es ambiguo ya que pueden tener muchas interpretaciones, por lo que no es el adecuado para dar órdenes precisas a una máquina.

Lenguaje máquina



Es el lenguaje nativo de las computadoras. Está compuesto por bits (0s y 1s) y es el único que la máquina puede entender directamente.

Este lenguaje es preciso porque no admite ambigüedades, pero es difícil para los humanos ya que es complejo escribir instrucciones solo con 0 y 1.

Aunque el lenguaje máquina es la base de todo, no podemos programar directamente en lenguaje máquina.

Tipos de lenguaje

Lenguaje de programación

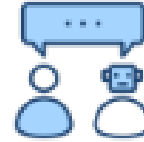
Es un lenguaje creado para que los humanos puedan dar instrucciones a las computadoras de manera comprensible, y que luego puedan ser traducidas al lenguaje máquina. Por tanto, actúa como puente entre el lenguaje natural y el lenguaje máquina

Cada lenguaje de programación tiene unas reglas claras y una sintaxis específica.

Según la cercanía a la máquina podemos clasificar los lenguajes de programación en lenguajes de bajo, medio y alto nivel.

Los lenguajes de bajo nivel son los más cercanos a la máquina, por eso son también los más complejos. Por ejemplo, el lenguaje ensamblador.

Los lenguajes de alto nivel son los que se encuentran más alejados de la máquina y más cercanos al lenguaje humano, y se enfocan en facilitar la programación. Ejemplos: **Python, Java, JavaScript, Ruby.**



MONITOR FOR 6802 1.4

9-14-80 TSC ASSEMBLER PAGE 2

```
C000                                ORG    ROM+$0000 BEGIN MONITOR
C000 8E 00 70  START  LDS     #STACK

*****
* FUNCTION: INITA - Initialize ACIA
* INPUT: none
* OUTPUT: none
* CALLS: none
* DESTROYS: acc A

0013          RESETA EQU    %00010011
0011          CTLREG EQU    %00010001

C003 86 13          INITA  LDA A  #RESETA  RESET ACIA
C005 B7 80 04              STA A  ACIA
C008 86 11              LDA A  #CTLREG  SET 8 BITS AND 2 STOP
C00A B7 80 04              STA A  ACIA

C00D 7E C0 F1              JMP    SIGNON  GO TO START OF MONITOR
```

Tipos de código

En el proceso de desarrollo de software, el código pasa por diferentes etapas antes de ser ejecutado por una computadora:

Código fuente: es un conjunto de instrucciones escritas en un lenguaje de programación determinado.



Código objeto: Es el código generado por un compilador tras traducir el código fuente. Está escrito en un formato intermedio, en lenguaje máquina, pero no es aún un programa completo.

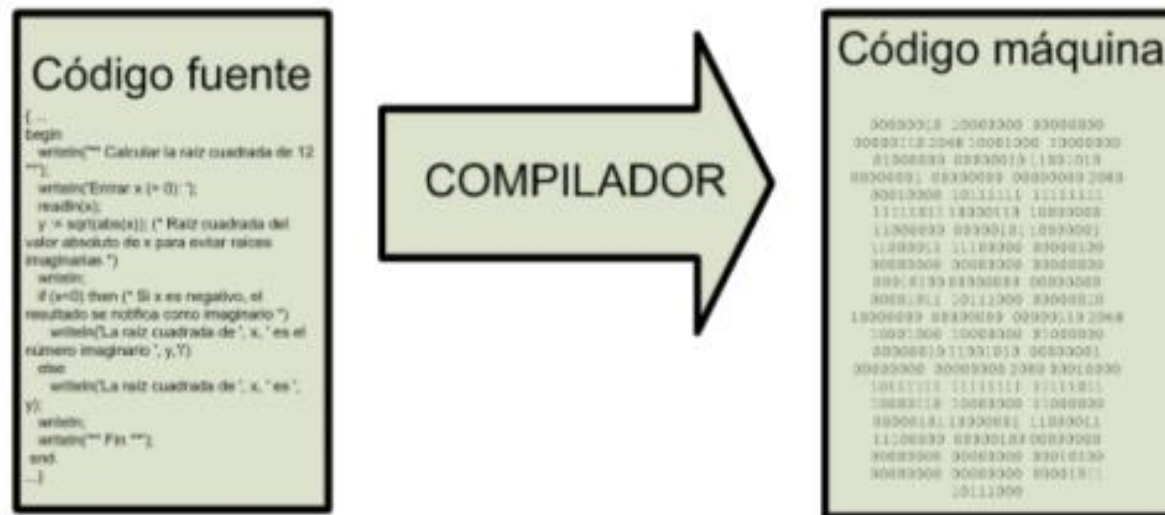
Código ejecutable: Es el archivo final que puede ser ejecutado directamente por la máquina, resultado de enlazar nuestro código objeto con las librerías necesarias.

Este código ya es nuestro programa ejecutable.

Es el archivo final que puede ser ejecutado directamente por la máquina y es específico para una plataforma concreta: Windows, Linux, Mac OS, o cierto sistema Hardware.

El lenguaje de programación Java

Los lenguajes de programación de alto nivel son más fáciles de entender y utilizar por los programadores, pero requieren de un compilador que traduzca ese código a un formato que la computadora pueda procesar y ejecutar.



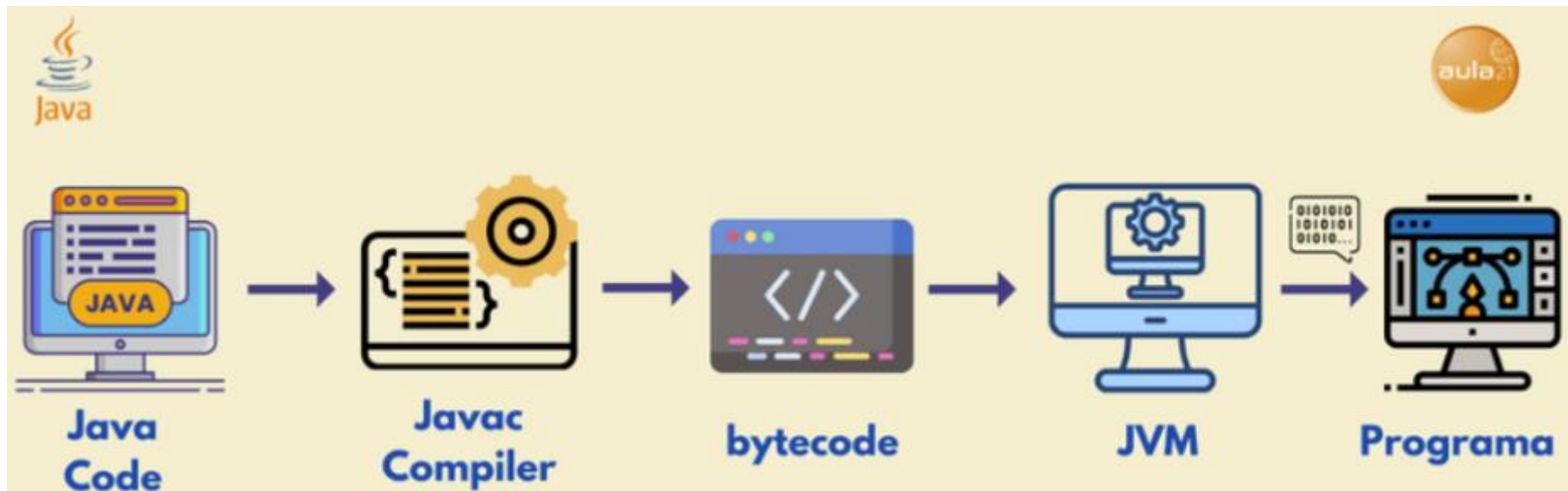
Java es un lenguaje que destaca por su portabilidad, lo que significa que puede funcionar en cualquier sistema operativo.

Cuando escribes código en Java, éste se convierte en un formato intermedio llamado **bytecode** después de ser compilado. Luego, ese bytecode es interpretado y ejecutado por la **Máquina Virtual de Java (JVM)**. Esto permite que los programas escritos en Java se puedan ejecutar en cualquier plataforma que tenga una JVM, sin importar cuál sea el sistema operativo.

El lenguaje de programación Java

Un programa Java se compone de uno o más archivos de código fuente con la extensión `.java`, que contienen las instrucciones del programador. Estos archivos se compilan con la herramienta **javac**, generando archivos con la extensión `.class`, los cuales contienen el bytecode. Dichos archivos se pueden ejecutar en cualquier sistema que tenga una JVM.

En resumen, Java es un lenguaje que compila su código a bytecode para ser ejecutado en una JVM, lo que permite su funcionamiento en cualquier sistema operativo, y cuenta con una gran cantidad de bibliotecas que facilitan el desarrollo de aplicaciones.



Fases de desarrollo de un programa

Ya sabemos cuáles son las fases de desarrollo de un programa:

1º Análisis ¿Qué me ha pedido el cliente? ¿Cuál es el problema?

2º Diseño ¿Cómo vamos a resolver el problema?

3º Codificación implementar, documentar, probar,...

4º Mantenimiento correcciones y evolutivos

Hemos llegado ya a la fase de Codificación que consiste en:

- Picar código
- Depurar
- Organizar agrupaciones de código
- Testear

Podríamos programar simplemente picando código en un bloc de notas, como hacían los primeros programadores en Java, pero no sería productivo, para mejorar la productividad se utilizan los “**Entornos integrados de Software**” (IDE) que son herramientas visuales que nos ayudan en las tareas de desarrollo.

Nosotros usaremos **Eclipse**, aunque no es el único.

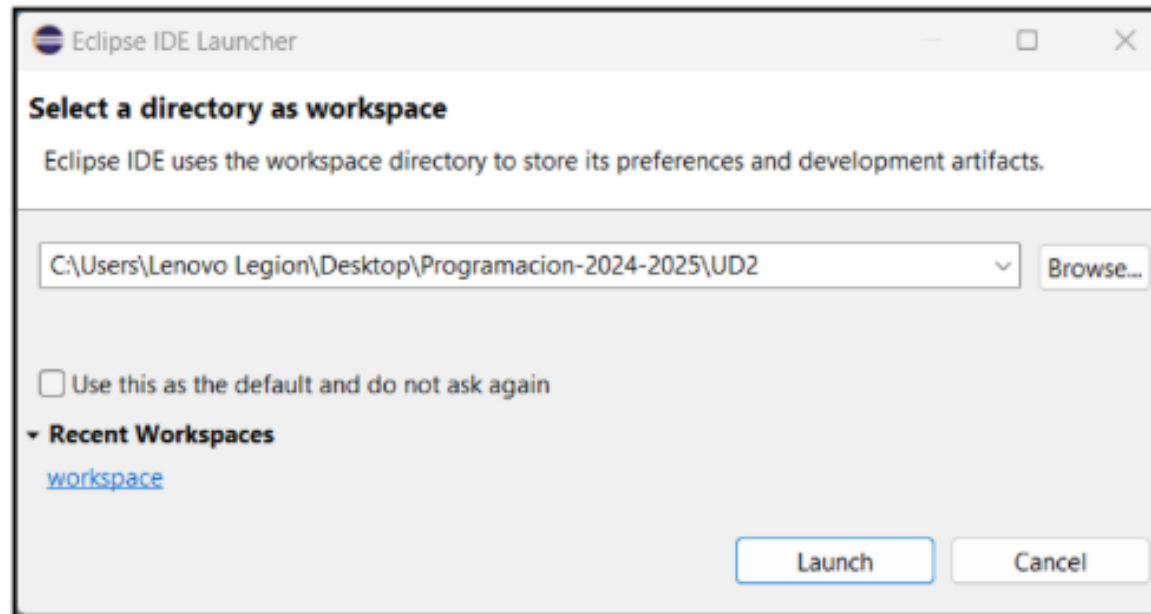


Estructura de un programa

1. El primer proyecto en Java

Una vez instalado Eclipse, cuando lo ejecutemos por primera vez, aparecerá un diálogo donde seleccionaremos la carpeta en la que vamos a almacenar nuestros proyectos “**workspace o Espacio de trabajo**”.

Podemos crear una carpeta donde crear todos los proyectos del curso o hacerlo por temas. Si indicamos una carpeta que no existe, el propio Eclipse la creará.



Después de crear la carpeta aparecerá una página de Bienvenida que podemos cerrar directamente.

Estructura de un programa

Crear proyecto

Vamos a crear nuestro **primer programa en Eclipse**, podemos hacerlo de varias maneras:

1º directamente desde el menú de la izquierda: *“Create a Java Project”*

2º Menú superior *File >New>Java Project*

Nombre del proyecto

El nombre que le demos al proyecto debe ser **descriptivo y no ser muy largo, no debemos espacios en blanco, ñ ni símbolos raros, lo habitual es que empiece por letra en mayúscula**. Dejamos las opciones que haya en el diálogo con sus valores por defecto y pulsamos *“Finish”*.

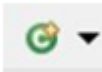
Una vez creado, a la izquierda en la ventana ***“Package Explorer”*** (explorador de paquetes) veremos el nombre de nuestro proyecto.

Estructura de un programa

Crear la clase

El segundo paso será crear una Clase dentro de nuestro proyecto, ya veremos más adelante qué es una clase, ahora lo único que debemos saber es que cualquier programa de Java necesita tener al menos una clase.

También ahora tenemos varios sitios desde los que poder crear una clase:

- Poniéndonos sobre el proyecto, botón derecho del ratón New>Class
- Menú superior: File>New>Class
- Desde el icono 

Estructura de un programa

Nombre de la clase

En el diálogo que aparece debemos definir el nombre de la clase, suele escribirse **todo en minúsculas con la inicial en mayúsculas**.

Dejamos todo por defecto y marcamos por comodidad las opciones “public static void main(String[] args)” y “Generate comments”.

Pulsamos el botón **Finish** y ya tendremos creada nuestra primera clase.

Para poder ejecutar un programa java, debe existir al menos una clase con un método **main**. Este método es el que buscará el programa para empezar su ejecución, en él se incluyen las sentencias (órdenes) del programa principal.

```
public class Ejemplo {  
  
    /**  
     * @param args  
     */  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
    }  
}
```

Estructura de un programa

Comentarios

Si queremos poner comentarios que expliquen el funcionamiento del programa o código que no queremos que se tenga en cuenta al ejecutar, escribiremos :

Si es una sola línea --> // al principio de la línea

Si son varias líneas--> /* */

```
// Comentario de una sola línea

/*
 * Comentario de varias líneas
 */
```

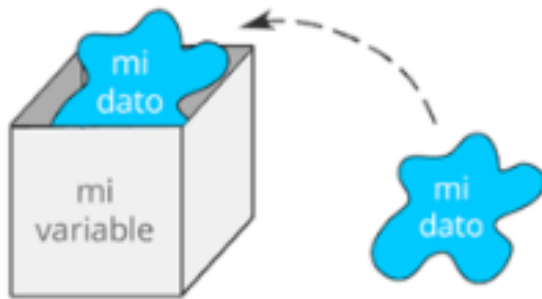
A tener en cuenta:

- Las sentencias (órdenes) finalizan con ;
- Java es sensible a mayúsculas y minúsculas, no es lo mismo system que System
- Las tabulaciones no son obligatorias, pero nos ayudan a que el código sea más legible.
- Evitar utilizar líneas de código de más de 80 caracteres
- Las líneas de código o sentencias se agrupan en un bloque que delimitado con }

Estructura de un programa

2. Las variables

Las **variables** en Java son espacios que se reservan en la memoria RAM del ordenador para almacenar un tipo de dato concreto.



- En un programa Java, para poder utilizar una variable antes la debemos:
- **Declarar:** definir el tipo de dato para reservar su espacio en memoria
- **Inicializar:** asignar un valor inicial

Estructura de un programa

2.1. Características

Las variables tienen una serie de características importantes:

- **Poseen un nombre:** Esto le permite al programa acceder al valor que tiene almacenado en la memoria. Este nombre, debe ser un identificador válido.
- **Contienen un tipo de dato:** se debe especificar qué tipo de dato va a almacenar. Pueden ser números enteros, decimales, caracteres de tipo texto, objetos, etc.
- El nombre de las variables se suele escribir en minúsculas o si está formado por varias palabras utilizar la regla camel case (la primera letra de cada palabra en mayúscula, excepto la primera).
- Deben comenzar por una letra (también lo pueden hacer por los caracteres \$ o _, pero no es aconsejable).
- No hay límite máximo de caracteres
- No puede emplearse el identificador de una variable o cualquier otro elemento del código fuente del programa para otro ya existente en el mismo bloque.

`int altura`



`int alturaEdificio`



`int AltURA`



`int ALTURA`



Estructura de un programa

3. Constantes

Una **constante** hace referencia a un valor que no puede modificarse a lo largo de la ejecución del programa (*por ejemplo, PI en el cálculo del área de un círculo*)

Las constantes se definen igual que cuando se declaran las variables y se les asigna un valor.

- La palabra reservada **final** es la que impide que se modifique el valor almacenado.
- Por convenio los identificadores asociados a constantes se ponen en **Mayúsculas**
- El signo “_” se acepta en cualquier identificador, pero por convención sólo se emplea para separar palabras en el nombre de las constantes (por convención las constantes se escriben completamente en mayúsculas y esto dificulta la separación de los nombres).

Por ejemplo: para un valor constante que almacena un valor numérico que representa el número máximo de alumnos con los que trabaja el programa se podría declarar el identificador MAXIMO_NUMERO_ALUMNOS

```
final double PI=3.141592;
```

```
final int MAX_NUMERO_ALUMNOS=30;
```

Estructura de un programa

4. Palabras reservadas

En Java existe una serie de palabras reservadas que no pueden utilizarse como identificadores de variables o constantes

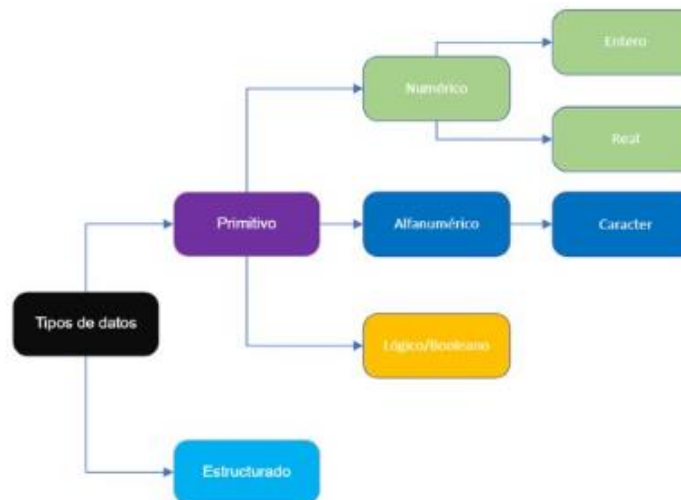
*Por ejemplo: la palabra **double** se utiliza para definir un tipo de dato real y la palabra **for** se emplea para construir un tipo determinado de bucle, por tanto, no se puede utilizar para identificar una variable o una constante.*

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Estructura de un programa

5. Tipos de datos

Cualquier lenguaje de programación incluye una serie de tipos de **datos primitivos**, es decir, incorporados por el propio lenguaje. También tenemos los tipos de **datos estructurados**, se construyen a través de clases y objetos (ya los veremos más adelante).



Tipos de datos primitivos

Numéricos	
Enteros	byte, short, int, long
Reales	float, double
Carácter	char
Lógico	boolean

Estructura de un programa

5.1. Números enteros

Tenemos 4 tipos básicos para representar los números enteros (con signo) en función del espacio que ocupan en memoria:

Tipo	Espacio en memoria	Valor mínimo	Valor máximo
byte	8 bits	-128	127
short	16 bits	-32768	32767
int	32 bits	-2^{31}	$2^{31}-1$
long	64 bits	-2^{63}	$2^{63}-1$

Por defecto los literales enteros son del tipo **int**.

Un literal entero es del tipo **long** si va acompañado del sufijo L o l, suele usarse L para que no se confunda con el número 1:

Ejemplo: 128546L

Si en una operación con números enteros, sobrepasamos el valor máximo, el programa no nos avisará del error, pero nos devolverá un valor erróneo:

Por ejemplo, si trabajamos con datos tipo byte: $127+1=-128$

Estructura de un programa

5.2. Números reales

Tipo de dato	Espacio en memoria	Mínimo (valor absoluto)	Máximo (valor absoluto)	Dígitos significativos
<u>float</u>	32 bits	1.4×10^{-45}	3.4×10^{38}	6
<u>double</u>	64 bits	4.9×10^{-324}	1.8×10^{308}	15

Por defecto los literales reales representan valores de tipo **double**.

Para representar un dato de tipo **float**, lo acompañaremos del sufijo F o f (si no el compilador dará error)

Ejemplo: 123.45f

Estructura de un programa

5.3. Caracteres

Tipo de dato	Espacio en memoria	Codificación
<u>char</u>	16 bits	UNICODE

Unicode es una codificación de caracteres de 16 bits que soporta la mayoría de los idiomas del mundo.

Los valores van entre comillas simples: 'a'

Se pueden usar en expresiones aritméticas

Estructura de un programa

5.4. Cadenas de caracteres. String

Tipo de dato	Espacio en memoria	Codificación
<u>char</u>	16 bits	UNICODE

La clase **String**, no es un tipo primitivo, sino una clase predefinida pero se usa habitualmente.

El texto va entre comillas dobles: “**Hola**”

Una cadena con un solo carácter no es un char.

Estructura de un programa

5.5. La tabla ASCII

La tabla ASCII (American Standard Code for Information Interchange) es una forma de representar caracteres como letras, números y símbolos en el ordenador. En Java, la tabla ASCII puede ser muy útil por varias razones:

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Estructura de un programa

5.5. La tabla ASCII

- **Manipulación de Caracteres:** cada carácter tiene un valor numérico en la tabla ASCII. Por ejemplo, la letra 'A' tiene el valor 65, si queremos convertir una letra minúscula a mayúscula, podemos restar 32 al valor ASCII de la letra minúscula.
- **Comparación de Caracteres:** podemos comparar caracteres fácilmente usando sus valores ASCII. Esto es útil para ordenar palabras alfabéticamente o para verificar si un carácter es una letra, un número o un símbolo.
- **Control de Entrada y Salida:** Algunos caracteres ASCII son caracteres de control, como el salto de línea (valor 10) o el tabulador (valor 9). Estos caracteres son esenciales para formatear la salida en la consola o en archivos.

Ejemplo: si ejecutamos el código:

```
char letra_a = 'a';  
char letra_a_ascii = 97;  
System.out.println("char='a' es " + letra_a);  
System.out.println("char=97 es " + letra_a_ascii);
```

La consola muestra:

```
char='a' es a  
char=97 es a
```

Estructura de un programa

5.6. Secuencias de escape

Las cadenas de texto, que están representadas por la clase **String** en Java es una secuencia de caracteres. Estos caracteres pueden ser letras, números, signos de puntuación, etc. Esta cadena debe estar entre comillas:

```
String saludo="Hola, soy Clara";
```

Pero ¿qué hacemos si necesitamos crear una cadena que contenga comillas?

```
String saludo="Hola, soy \"Clara\"";
```

Estructura de un programa

5.6. Secuencias de escape

Parece que al compilador no le gusta, ¿cuál es el problema? El compilador espera que las cadenas de caracteres estén envueltas entre comillas. por tanto, cada vez que el compilador ve ", espera otras " y que el contenido entre ellas sea el texto de una cadena. En nuestro caso, las comillas alrededor la palabra "Clara" está dentro de otras comillas y eso el compilador no lo entiende. Para poder usar unas comillas dentro de otras en Java debemos usar **caracteres de escape**. Esto se logra usando el símbolo de la barra invertida: \ delante del caracter que debe "escaparse".

Por ejemplo, \" es una secuencia de control para mostrar comillas en la pantalla

A continuación, tenemos una tabla con las diferentes secuencias de escape que podemos encontrar en Java:

Secuencia de Escape	Descripción
\n	Salto de línea
\t	Tabulador
\\	Diagonal Inversa \
\"	Comillas Dobles
\'	Comilla Simple
\r	Retorno de Carro (Solo en modo Administrador)
\b	Borrado a la Izquierda (Solo en modo Administrador)

Estructura de un programa

5.7. Datos Lógicos o Booleanos

Tipo de dato	Espacio en memoria	Valores
<u>boolean</u>	1 bit	Verdadero o falso

Estructura de un programa

6. Salidas de datos

La salida de datos es la impresión de mensajes por pantalla que nos permiten mostrar o solicitar información al usuario.

Para imprimir por pantalla, se usa la clase **System**, el atributo **out**, y sus métodos **println()** o **print()**:

```
System.out.print("Mensaje");  
System.out.println("Mensaje");
```

La diferencia entre **print()** y **println()** es que **println()** muestra el texto y luego deja un salto de línea, mientras que **print()** no lo hace.

Por ejemplo, el código:

```
System.out.print("hola");  
System.out.print("Adiós");
```

al ejecutarse muestra:

```
|holaAdiós
```

Mientras que con:

```
System.out.println("hola");  
System.out.println("Adiós");
```

tendremos:

```
|hola  
|Adiós
```

Estructura de un programa

6.1. printf. Salidas de datos con formato

Printf en Java, nos permite aplicar formatos específicos a una salida de datos por pantalla.

La **f** en **printf** da el significado de *formateado* y nos permite especificar cómo deben mostrarse los datos, *por ejemplo, cuántos decimales mostrar para un número o cómo alinear un texto*. Es igual que cuando le damos formato a un texto en cualquier procesador de texto, como Word.

La sintaxis general es:

```
System.out.printf(String del formato, datos);
```

- **String del formato:** es una cadena de caracteres que contiene texto fijo y los **especificadores de formato**.
- **datos:** son los valores que se insertarán en la cadena de formato.

Especificadores de formato

Los especificadores de formato indican el tipo de dato y cómo debe mostrarse. Algunos de los más habituales son:

Estructura de un programa

6.1. printf. Salidas de datos con formato

Especificadores de formato

Los especificadores de formato indican el tipo de dato y cómo debe mostrarse. Algunos de los más habituales son:

%d	Números enteros (int)
%f	Números decimales (float,double)
%s	Cadena de texto (String)
%c	Caracteres (char)
%%	Para imprimir el símbolo %

¿Cómo lo usamos?

Aquí tenemos un ejemplo básico de cómo mostrar por pantalla la información utilizando printf:

```
int edad = 21;
String nombre = "Mario";
double altura = 1.75;

System.out.printf("%s tiene %d años y mide %f", nombre,edad, altura);
```

El resultado es: |Mario tiene 21 años y mide 1,750000

Estructura de un programa

6.1. printf. Salidas de datos con formato

Control de decimales

Podemos controlar la cantidad de decimales que vamos a mostrar utilizando un punto y a continuación el número de decimales, *por ejemplo, si queremos mostrar solo 2 decimales lo indicaremos pondremos: %.2f*:

```
int edad = 21;  
String nombre = "Mario";  
double altura = 1.75;  
  
System.out.printf("%s tiene %d años y mide %.2f", nombre, edad, altura);
```

El resultado ahora es:

```
Mario tiene 21 años y mide 1,75
```

Estructura de un programa

6.1. printf. Salidas de datos con formato

Alineación y ancho de campo

Podemos controlar el ancho mínimo que ocupará un valor en la pantalla, lo que nos ayudará a alinear columnas. Para especificar el ancho, pondremos un número antes del especificador.

```
System.out.printf("%10s %10s\n", "Alumno", "Nota");  
System.out.printf("%10s %10.2f\n", "Mario", 5.20);  
System.out.printf("%10s %10.2f", "Sara", 9.15);
```

El resultado ahora es:

Alumno	Nota
Mario	5,20
Sara	9,15

Si queremos alinear a la izquierda, añadiremos un **guión** - antes del número:

```
System.out.printf("%-10s %10s\n", "Alumno", "Nota");  
System.out.printf("%-10s %10.2f\n", "Mario", 5.20);  
System.out.printf("%-10s %10.2f", "Sara", 9.15);
```

Y tendremos:

Alumno	Nota
Mario	5,20
Sara	9,15

Estructura de un programa

7. Entrada de datos

La clase **Scanner**, es una clase de la API de java, que se utiliza para leer los datos que el usuario introduce por teclado. Pertenece al paquete **java.util.Scanner**, por tanto, para poder utilizarla tendremos que poner **import java.util.Scanner** en la parte superior de la clase, debajo del paquete.

Lo primero que debemos hacer es crear un objeto de la clase Scanner:

```
Scanner entrada = new Scanner (System.in) ;
```

Una vez que ya hemos creado el objeto al cual hemos llamado entrada, podemos acceder a diferentes métodos de la clase Scanner para leer cualquier tipo de dato:

next() : realiza una lectura hasta que encuentra el primer espacio

nextLine() : lee una línea completa de caracteres alfanuméricos

nextDouble() : lee números decimales de doble precisión

nextFloat() : lee números decimales de simple precisión

nextInt() : lee números enteros

nextBoolean() : lee un dato lógico (true/false)

Estructura de un programa

7.1. Ejemplos

Ejemplo 1

Vamos a crear un programa que pida al usuario su nombre y edad y luego muestre un saludo.

```
public static void main(String[] args) {  
  
    String nombre;  
    int edad;  
    Scanner leer =new Scanner(System.in);  
    System.out.println("Introduce tu nombre");  
    nombre=leer.nextLine();  
    System.out.println("Introduce tu edad");  
    edad=leer.nextInt();  
    System.out.println("Hola "+nombre+" felices "+edad);  
    leer.close();  
  
}
```

Estructura de un programa

7.1. Ejemplos

Ejemplo 2

En este caso el programa sería correcto y funcionaría correctamente. Pero ¿qué pasaría si pedimos los datos en el orden contrario, es decir, en caso de pedir primero la edad y luego el nombre?

```
public static void main(String[] args) {  
  
    String nombre;  
    int edad;  
    Scanner leer =new Scanner(System.in);  
    System.out.println("Introduce tu edad");  
    edad=leer.nextInt();  
    System.out.println("Introduce tu nombre");  
    nombre=leer.nextLine();  
    System.out.println("Hola "+nombre+" felices "+edad);  
    leer.close();  
  
}
```

Estructura de un programa

7.1. Ejemplos

Ejemplo 2

En este caso el programa sería correcto y funcionaría correctamente. Pero ¿qué pasaría si pedimos los datos en el orden contrario, es decir, en caso de pedir primero la edad y luego el nombre?

Ahora el programa pide la edad del usuario, pero cuando nos pide el nombre, no podemos escribirlo y el programa continúa su ejecución mostrando la frase correspondiente al nombre y a la edad. Además, nos dice que el programa ha finalizado con éxito, pero realmente no ha funcionado como esperábamos.

¿Qué ha pasado?

Para entenderlo vamos a ver primero el concepto de **buffer**.

El **buffer**, es una parte de la memoria del ordenador que se reserva para almacenar datos que provienen desde los periféricos de entrada como por ejemplo un teclado. Esto quiere decir que los datos que se teclean hacen el siguiente recorrido: Se escribe un dato y al pulsar ENTER, este se envía a la memoria del ordenador.

Un buffer, recibirá por un lado el dato enviado y por otro la pulsación del ENTER en un código Ascii.

Estructura de un programa

7.1. Ejemplos

Ejemplo 2

Una vez que el buffer recibe el dato, lo pasa a la variable correspondiente.

Cuando introducimos la edad y pulsamos ENTER el buffer está recibiendo 2 datos: Por un lado, la edad, por ejemplo, el número 25.

Y por otro, esta recibiendo un “\n” del ENTER.

Este símbolo, \n, es considerado un valor tipo String, es decir un texto. Esto quiere decir que:

El buffer enviará a la variable numérica el valor 25 y se guardará el carácter “\n” . Por ese motivo al invocar al método **nextline()**; en la variable **nombre** se almacena un carácter en blanco, ya que está leyendo el salto de línea que se ha almacenado antes en dicho buffer.

Estructura de un programa

7.1. Ejemplos

¿Cómo solucionamos el problema?

Para solucionar este problema debemos hacer una **limpieza de buffer** antes de pedirle el nombre al usuario. Esto lo hacemos con el método **nextLine()**; de forma que el código quedaría de la siguiente manera:

```
public static void main(String[] args) {  
  
    String nombre;  
    int edad;  
    Scanner leer =new Scanner(System.in);  
    System.out.println("Introduce tu edad");  
    edad=leer.nextInt();  
    leer.nextLine();  
    System.out.println("Introduce tu nombre");  
    nombre=leer.nextLine();  
    System.out.println("Hola "+nombre+" felices "+edad);  
    leer.close();  
  
}
```

Estructura de un programa

7.2. Lectura de un carácter

La clase **Scanner** NO CONTIENE un método *nextChar()* para leer un dato de tipo **char** desde teclado.

Si necesitamos leer un solo carácter desde teclado utilizaremos **System.in.read()**;

El método `System.in.read()` puede lanzar una **excepción** por lo que hay que añadir en la cabecera del método **main** lo siguiente: **throws IOException**

```
import java.io.IOException;

/**
 *
 */
public class LeerCaracter {

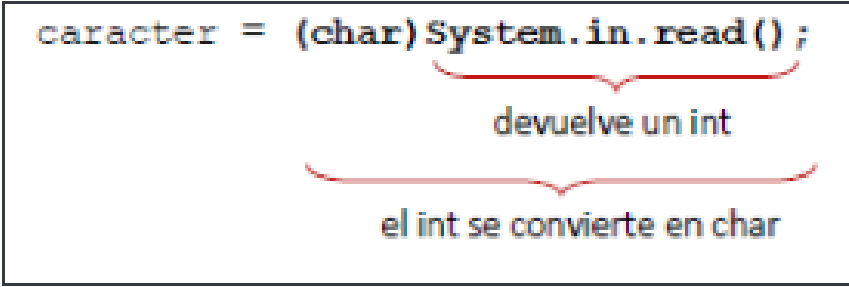
    /**
     * @param args
     * @throws IOException
     */
    public static void main(String[] args) throws IOException {
        // TODO Auto-generated method stub
        char character;
        System.out.print("Introduce un carácter: ");
        character = (char)System.in.read();
        System.out.println("Se ha introducido: " + character);
    }
}
```

Estructura de un programa

7.2. Lectura de un carácter

El método **read()** devuelve un entero que corresponde al código ASCII del carácter introducido por teclado. Por eso es necesario realizar la conversión a tipo **char** antes de hacer la asignación a la variable carácter.

```
caracter = (char) System.in.read();
```



The diagram illustrates the type conversion in the code. A red bracket under the `System.in.read()` part of the code is labeled "devuelve un int". A second red bracket under the entire right-hand side expression `(char) System.in.read()` is labeled "el int se convierte en char".

Operadores

1. Operadores

Los **operadores** son elementos fundamentales que nos permiten realizar operaciones y manipular datos y variables. Estas operaciones pueden abarcar desde simples cálculos matemáticos hasta evaluaciones de condiciones lógicas y manipulación de datos a nivel de bits.

Son símbolos o palabras clave que se utilizan para llevar a cabo estas acciones de manera eficiente. Existen los siguientes tipos de operadores:

- **De asignación:** permiten asignar valores a variables.
- **Operadores aritméticos:** son símbolos especiales que se utilizan para realizar operaciones matemáticas entre datos de tipo numérico y dan como resultado otro valor de tipo numérico. *Ejemplo: división (/); producto (*); suma (+); resta (-); módulo (%).*
- **Operadores de comparación o relacionales:** Permiten la comparación entre datos del mismo tipo y dan como resultado dos valores posibles: Verdadero o Falso. *Ejemplo: igual a (=); menor que (<); distinto (!=).*
- **Operadores lógicos:** Posibilitan la evaluación lógica de dos expresiones de tipo lógico. Dan como resultado uno de dos valores posibles: Verdadero o Falso. *Ejemplo: negación (!); conjunción Y (&&); disyunción O (||).*